**NAME – SUBASH G**
**REG NO – 192572150**

**COURSE NAME – Operation System**
**COURSE CODE – CSA0408**

# Lab Observations

**EXPERIMENT - 5**

Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

**PROGRAM:**

```
#include <stdio.h>

struct priority_scheduling
{
    char process_name;
    int burst_time;
    int waiting_time;
    int turn_around_time;
    int priority;
};

int main()
{
    int number_of_process;
    int total = 0;
    int ASCII_number = 65;
    int position;

    float average_waiting_time;
    float average_turnaround_time;

    struct priority_scheduling temp_process;

    printf("Enter the total number of Processes: ");
```

```
scanf("%d", &number_of_process);

struct priority_scheduling process[number_of_process];

printf("\nPlease Enter the Burst Time and Priority of each process:\n");

for (int i = 0; i < number_of_process; i++)
{
    process[i].process_name = (char)ASCII_number;

    printf("\nEnter the details of the process %c\n",
           process[i].process_name);

    printf("Enter the burst time: ");
    scanf("%d", &process[i].burst_time);

    printf("Enter the priority: ");
    scanf("%d", &process[i].priority);

    ASCII_number++;
}

/* Sort processes according to priority */
for (int i = 0; i < number_of_process; i++)
{
    position = i;

    for (int j = i + 1; j < number_of_process; j++)
    {
        if (process[j].priority > process[position].priority)
            position = j;
    }

    temp_process = process[i];
    process[i] = process[position];
    process[position] = temp_process;
}

/* Calculate waiting time */
process[0].waiting_time = 0;

for (int i = 1; i < number_of_process; i++)
```

```
    {
        process[i].waiting_time = 0;

        for (int j = 0; j < i; j++)
        {
            process[i].waiting_time += process[j].burst_time;
        }

        total += process[i].waiting_time;
    }

    average_waiting_time =
        (float)total / number_of_process;

    /* Calculate turnaround time */
    total = 0;

    printf("\n\nProcess_name\tBurst Time\tPriority\tWaiting Time\tTurnaround
Time\n");

    for (int i = 0; i < number_of_process; i++)
    {
        process[i].turn_around_time =
            process[i].burst_time + process[i].waiting_time;

        total += process[i].turn_around_time;

        printf("\t%c\t\t%d\t\t%d\t\t%d\t\t%d\n",
               process[i].process_name,
               process[i].burst_time,
               process[i].priority,
               process[i].waiting_time,
               process[i].turn_around_time);
    }

    average_turnaround_time =
        (float)total / number_of_process;

    printf("\nAverage Waiting Time : %f",
           average_waiting_time);

    printf("\nAverage Turnaround Time : %f\n",
```

```
            average_turnaround_time);

    return 0;
}
```

**OUTPUT:**

